The Code Is There. Does the Agent Understand It?

Evaluating the Effect of Architecture Documentation on Autonomous Coding Agents

1. Hypotheses

This experiment investigates whether providing an architecture document improves an autonomous coding agent's performance when modifying an existing software project. **RQ1 – Correctness:** Does the agent finish tasks correctly more often with the document? **RQ2 – Design fit:** Does it help preserve existing tests and project design? **RQ3 – Effort:** Does it change task time or tool-call count?

H1 – Correctness. I expected more correct completions with the architecture document because it explains component responsibilities, relationships, and extension points, helping the agent identify where changes belong and how new functionality should integrate.

H2 – Design fit. I expected improved design fit and fewer regressions. Understanding abstractions, component boundaries, and constraints should help the agent make changes consistent with the project's structure.

H3 – Effort. I expected the document to reduce codebase exploration, unnecessary tool calls, and completion time by providing an architectural overview before implementation.

2. Experimental Setup

2.1 Repository and baseline

The experiment used **TinyDB**, a Python database library (upstream: https://github.com/msiemens/tinydb). The prepared snapshot was commit **eae18d1**, “Initial commit: cleaned TinyDB snapshot for AutoSE experiment.” The same snapshot was used for both conditions. Environment: local Windows machine, Python 3.14.7, pytest 9.1.0.

The baseline had **228 passed, 0 failed, 0 skipped** tests and **93% coverage**. Removed paths are listed in `experiment/removed_files.txt`; these include documentation-related paths such as `docs/`, static assets, templates, themes, and documentation content.

2.2 Tasks

Task	Description	Main area
T1	Fix generator exhaustion in one_of() query operation.	Query
T2	Add a between() query operation.	Query
T3	Add gzip-compressed JSON storage through CompressedJSONStorage, integrated with TinyDB and CachingMiddleware.	Storage / middleware

2.3 Conditions and agent configuration

Condition A: source code and task specification only. **Condition B:** the same source code and task specification plus `ARCHITECTURE.md`, with the instruction “Read ARCHITECTURE.md before you start.” Three tasks were tested under both conditions, with two trials per task and condition (3 × 2 × 2 = **12 runs**). Each run used a fresh repository copy and fresh Claude Code session.

Parameter	Configuration
Agent / model	Claude Code (Claude Desktop – Code tab) / Claude Opus 5.5
Model identifier / version	claude-opus-5-5 / Claude Code 2.1.317
Effort / permission	Medium / Auto
Machine / OS	Same local Windows machine for all runs
Python / pytest	3.14.7 / 9.1.0
Time limit	30 minutes per run
Completion instruction	“When you are done, make sure the existing tests still pass.”
Agent clarification	“Use your best judgement.”
Temperature	Not exposed in the interface

3. Architecture Document

The architecture document was created to give the coding agent a structured understanding of TinyDB before modifying it. It was developed with LLM assistance and checked against the source code. Inspected files included `tinydb/database.py`, `table.py`, `storages.py`, `middlewares.py`, `queries.py`, and `utils.py`.

It describes seven areas: TinyDB/database coordination, Table operations, Storage, Middleware, Query, Operations, and Utilities. It includes a component relationship diagram, three Architecture Decision Records (ADRs), and five developer constraints. ADRs without historical evidence were labelled as inferred rather than confirmed historical decisions.

Time spent creating the architecture document: 3 hours.

3.1 Checked architecture claims

#	Claim checked	Source location	Result
1	Storage defines read() and write().	storages.py:36–69	Correct
2	TinyDB constructs selected storage.	database.py:70	Correct
3	TinyDB closes storage during lifecycle cleanup.	database.py (close / __exit__)	Correct
4	Middleware wraps storage.	middlewares.py:17–23	Correct
5	Middleware forwards attributes.	middlewares.py:63–73	Correct
6	CachingMiddleware buffers writes and flushes.	middlewares.py (cache / flush)	Correct
7	queries.py does not import table.py.	queries.py:19–23; table.py:15–17	Correct
8	Table manages query-result caching and invalidation.	table.py:278–295, 850–854	Correct
9	Query identity uses freeze() for stable representation.	queries.py; utils.py	Correct
10	Storage is an extension point for storage implementations.	storages.py:36–69	Correct

All 10 checked factual claims were correct against the inspected source locations. This verification was intended to reduce the risk of providing inaccurate architectural information to the agent.

4. Results

4.1 Run-level results

A run was Complete when all acceptance tests passed, Partial when at least one but not all passed, and Failed when none passed. Regressions count original baseline tests that passed before a run and failed afterward.

Run	Task	Cond.	Outcome	Accept.	Reg.	Fit	Min	Calls
T1_A_1	T1	A	Complete	4/4	0	3/3	1.49	9
T1_A_2	T1	A	Complete	4/4	0	3/3	1.93	9
T1_B_1	T1	B	Complete	4/4	0	3/3	1.97	9
T1_B_2	T1	B	Complete	4/4	0	3/3	1.75	11
T2_A_1	T2	A	Partial	1/4	0	3/3	1.22	8
T2_A_2	T2	A	Partial	1/4	0	3/3	1.12	10
T2_B_1	T2	B	Partial	1/4	0	3/3	1.46	10
T2_B_2	T2	B	Partial	1/4	0	3/3	1.17	7
T3_A_1	T3	A	Failed	0/4	0	3/3	2.22	10
T3_A_2	T3	A	Failed	0/4	0	3/3	1.72	7
T3_B_1	T3	B	Partial	3/4	0	3/3	2.58	10
T3_B_2	T3	B	Failed	0/4	0	3/3	1.72	8

4.2 Condition-level summary

Metric	Condition A: Code only	Condition B: Code + architecture
Runs	6	6
Complete runs / rate	2 / 33.33%	2 / 33.33%
Mean regressions	0.00	0.00
Mean design fit	3.00 / 3	3.00 / 3
Mean time	1.62 min	1.78 min
Mean tool calls	8.83	9.17
Acceptance tests passed	10 / 24	13 / 24
Acceptance pass rate	41.67%	54.17%

4.3 Additional effort and task-level results

Metric	Condition A	Condition B
Mean input tokens	797,583	859,625
Mean output tokens	7,575	6,163

Task	Condition A	Condition B
T1	8/8	8/8
T2	2/8	2/8
T3	0/8	3/8
Total	10/24	13/24

Condition B had a higher acceptance pass rate by 12.50 percentage points, but the complete rate was unchanged. It also used more input tokens, fewer output tokens, slightly more tool calls, and slightly more time on average. These measurements do not indicate an effort reduction.

5. Analysis

5.1 RQ1 – Correctness

Condition A passed 10/24 acceptance tests (41.67%), while Condition B passed 13/24 (54.17%), a difference of 12.50 percentage points. Both conditions completed two of six runs (33.33%). T1 and T2 had identical totals; the entire difference came from T3 (A: 0/8; B: 3/8). The improvement was task-specific, not consistent across all tasks.

T2 evaluator expectations included differences from TinyDB API conventions: `inclusive` versus `include_lower`/`include_upper`, and expectations about `remove()` return values. T2 scores are therefore not a pure measure of implementation quality. For T3, some runs used `GzipJSONStorage` while the evaluator expected `CompressedJSONStorage`; those tests failed at class lookup and did not assess compression behaviour.

Answer to RQ1: The architecture condition had a higher acceptance-test pass rate, but no increase in complete-run rate. The difference was limited to T3.

5.2 RQ2 – Design fit

All 12 runs scored 3/3 on design fit and recorded zero baseline regressions. The document produced no measurable improvement in these metrics. The maximum score in both conditions also creates a possible ceiling effect.

Answer to RQ2: No measurable difference was observed; both conditions achieved maximum design fit and zero regressions.

5.3 RQ3 – Effort

Condition A averaged 1.62 minutes and 8.83 tool calls. Condition B averaged 1.78 minutes and 9.17 tool calls. Mean input tokens were 797,583 for A and 859,625 for B; mean output tokens were 7,575 for A and 6,163 for B. The architecture document did not reduce measured time or tool calls.

Answer to RQ3: Condition B was slightly slower and used marginally more tool calls. This experiment provides no evidence of an efficiency benefit.

5.4 Selected run analysis

T3_A_1 – Code only. Without the architecture document, the agent modified `tinydb/storages.py` and `tinydb/__init__.py` to add gzip-compressed JSON storage, but named the class `GzipJSONStorage` instead of the expected `CompressedJSONStorage`. All four acceptance tests failed at class lookup, so they did not reach compression behaviour. The run had 250 regression tests passing, zero regressions, 3/3 design fit, 2.22 minutes, and 10 tool calls.

T3_B_1 – Code plus architecture. The agent read `ARCHITECTURE.md` and inspected storage implementation and public exports. It implemented and exported `CompressedJSONStorage` using the existing storage abstraction. Three of four acceptance tests passed; the remaining failure concerned rejecting plain, uncompressed JSON input. The run had 243 regression tests passing, zero regressions, 3/3 design fit, 2.58 minutes, and 10 tool calls.

Where it helped: T3_B_1 followed the storage extension point and expected class name, achieving 3/4 acceptance tests versus 0/4 in T3_A_1. **Where it made no difference:** T1 and T2 totals were identical, as were design-fit and regression measures. **Where it did not guarantee success:** T3_B_2 read the document but still used the wrong class name and passed 0/4. **Possible effort cost:** B had slightly higher average time, tool calls, and input tokens; this small experiment cannot establish that the document caused those differences.

6. Limitations

1. Small number of trials. Only two trials were performed per task and condition, giving six runs per condition. This is insufficient to establish a reliable general pattern.

2. Researcher involvement. The same researcher prepared the task specifications, design-fit checks, and architecture document, creating a possibility of framing bias.

3. Agent variability. Autonomous agents can produce different implementations for the same task. T3 runs selected different public class names, demonstrating run-to-run variation.

4. Evaluator contract limitations. Some T2 expectations did not align with TinyDB API conventions. T3 class-name mismatches prevented tests from reaching compression behaviour. Scores must be interpreted with these constraints in mind.

5. Limited design-fit measurement. Three binary checks per task were used. All runs received the maximum score, so the measure could not distinguish conditions and may have a ceiling effect.

6. Limited effort measurement. Time, tool calls, and available token usage were recorded, but temperature was not exposed. Token counts alone are not a complete measure of reasoning effort or computational cost.

7. Single project and model configuration. The experiment used one project, one agent, and one model configuration. Results may not generalize to other repositories, languages, agents, or models.

7. Conclusion

This experiment examined whether providing an architecture document improves an autonomous coding agent's ability to modify an existing software project. Condition B achieved a higher acceptance-test pass rate than Condition A (54.17% versus 41.67%), but the complete-run rate remained 33.33% in both conditions.

The improvement was entirely attributable to T3; T1 and T2 showed no difference. T3_B_1 demonstrated that the agent could use the architecture document to understand the storage extension point and implement the expected public class name. However, another architecture-informed run still selected an incorrect class name, showing that documentation alone does not guarantee successful implementation.

Both conditions achieved 3/3 design fit and zero regressions. Condition B was slightly slower and used marginally more tool calls, so no efficiency benefit was observed. Given the small number of trials, evaluator limitations, and agent variability, the findings are preliminary.

Overall conclusion: Architecture documentation showed a task-specific benefit for correctness, but this experiment did not establish a consistent improvement in task completion, design fit, or execution efficiency.